

Типовые ошибки при работе с Miri и Valgrind

- 1 Ошибка 1. Miri не запускается или выдаёт ошибки компиляции**
Miri может не запуститься, если код использует фичи, которые Miri не поддерживает, или если есть проблемы с установкой. Убедитесь, что Miri установлен через `rustup component add miri`, что используется актуальная версия Rust и что код не использует неподдерживаемые фичи (например, `inline assembly` или специфичные платформенные API).
- 2 Ошибка 2. Miri работает очень медленно**
Miri интерпретирует код, а не выполняет скомпилированный машинный код, поэтому выполнение в Miri значительно медленнее, чем обычное выполнение. Это нормально для Miri. Если выполнение занимает слишком много времени, попробуйте запустить Miri только на отдельных модулях или тестах, а не на всей программе.
- 3 Ошибка 3. Miri не находит проблему, которая есть в коде**
Miri может не найти проблему, если она проявляется только при определённых условиях или входных данных, которые не были протестированы. Убедитесь, что вы запускаете Miri с теми же входными данными, при которых проявляется проблема. Также помните, что Miri не поддерживает все фичи Rust и некоторые проблемы могут не обнаруживаться.
- 4 Ошибка 4. Ложные срабатывания Miri**
В некоторых случаях Miri может выдать предупреждение о проблеме, которая на самом деле не является ошибкой. Это может произойти при работе с нестандартными аллокаторами или при использовании фич, которые Miri не полностью понимает. Если вы уверены, что код корректен, но Miri выдаёт предупреждение, попробуйте запустить код под Valgrind для дополнительной проверки.
- 5 Ошибка 5. Valgrind не запускается или не работает**
Valgrind работает только на Linux и некоторых Unix-системах. На macOS поддержка ограничена, на Windows Valgrind не поддерживается. Убедитесь, что вы используете поддерживаемую систему и что Valgrind установлен правильно.
- 6 Ошибка 6. Valgrind не находит утечки в Rust-коде**
Valgrind может не найти утечки в безопасном Rust-коде, потому что Rust автоматически управляет памятью, и компилятор гарантирует, что память освобождается корректно. Valgrind полезен для анализа `unsafe`-кода, FFI или работы с C-библиотеками, где память может управляться вручную.
- 7 Ошибка 7. Отчёт Valgrind сложно читать**
Отчёты Valgrind могут быть длинными и содержать много информации. Сосредоточьтесь на секциях "ERROR SUMMARY" и "LEAK SUMMARY", которые показывают основные проблемы. Используйте опции `--leak-check=full` и `--show-leak-kinds=all` для получения подробной информации об утечках.
- 8 Ошибка 8. Valgrind находит проблемы в стандартной библиотеке**
Valgrind может выдать предупреждения о проблемах в стандартной библиотеке Rust или в системных библиотеках. Обычно это ложные срабатывания и их можно игнорировать. Сосредоточьтесь на проблемах в вашем коде, а не в стандартных библиотеках.